

Spark SQL

Sesión 7

Agenda

¿Qué veremos hoy?

1. **Introducción a Spark SQL** — historia, arquitectura y Catalyst Optimizer
2. **Estructura de datos** — DataFrames, Datasets y esquemas
3. **Consultas y análisis** — API SQL, DataFrame DSL, joins, window functions
4. **Ejemplos de análisis** — UDFs, EDA, optimización con `explain()`
5. **Casos de estudio** — Airbnb, Uber y Netflix

01

Introducción a Spark SQL

De Shark a Spark SQL: unificando SQL y procesamiento distribuido

¿Qué es Spark SQL?

Historia y motivación

- **2013 — Shark:** primer intento de SQL sobre Spark, construido sobre Hive; limitado por la arquitectura de Hive (Engle et al., 2013)
- **2014 — Spark SQL:** reescritura desde cero; desacopla el motor de ejecución del parsing SQL
- **2015 — Paper SIGMOD:** Armbrust et al. describen la arquitectura con **Catalyst** y **Tungsten**
- **2016–hoy:** evolución constante; integrado como módulo central de Apache Spark

Objetivo central: unificar en una sola API el procesamiento relacional (SQL) con las capacidades programáticas de Spark (Armbrust et al., 2015)

Componentes de Spark SQL

Una arquitectura en tres capas

Interfaces de consulta

- DataFrame API (DSL)
- Dataset API (tipado)

Catalyst Optimizer

- Análisis y resolución lógica
- Optimización basada en reglas y costos
- Generación de código (Whole-Stage CodeGen)

Tungsten Engine

- Gestión de memoria off-heap
- Ejecución vectorizada
- Cache-aware data structures

(Armbrust et al., 2015; Databricks, 2015a)

SparkSession

- Punto de entrada unificado desde Spark 2.0
- Reemplaza `SQLContext` e `HiveContext`

SparkSession: punto de entrada unificado

Evolución de las APIs

Versión	Punto de entrada	Alcance
Spark 1.x	<code>SparkContext</code>	Solo RDDs
Spark 1.x	<code>SQLContext</code>	SQL + DataFrames
Spark 1.x	<code>HiveContext</code>	SQL + Hive metastore
Spark 2.0+	<code>SparkSession</code>	Todo lo anterior unificado


```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MiApp") \
    .config("spark.sql.shuffle.partitions", "50") \
    .getOrCreate()
```

(Apache Spark, 2024a; Damji et al., 2020)

02

Estructura de Datos: DataFrames y Datasets

Tres APIs, una sola abstracción distribuida

Las tres APIs de Spark

RDD vs DataFrame vs Dataset

Característica	RDD	DataFrame	Dataset
Tipo	<code>RDD[T]</code>	<code>Dataset[Row]</code>	<code>Dataset[T]</code>
Tipado en compilación	Sí	No	Sí
Optimización Catalyst	No	Sí	Sí
Serialización Tungsten	No	Sí	Sí
Lenguajes	Scala, Java, Python, R	Todos	Scala, Java
API	Funcional	Relacional + funcional	Relacional + funcional

En Python solo existen **RDD** y **DataFrame** (= `Dataset[Row]`). Los Datasets tipados requieren el sistema de tipos de Scala/Java (Databricks, 2016)

DataFrame: Dataset [Row]

La abstracción central en PySpark

- Colección **distribuida** de filas con **esquema** conocido
- Evaluación **lazy**: las transformaciones se acumulan hasta una acción
- Columnas representadas como objetos `Column` — permiten expresiones

```
# Crear DataFrame desde CSV (inferencia de esquema)
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("ventas.csv")

df.printSchema()
# root
# |-- producto: string (nullable = true)
# |-- cantidad: integer (nullable = true)
# |-- precio: double (nullable = true)
```

(Apache Spark, 2024a; Databricks, 2024)

Definición explícita de esquemas

StructType y StructField

Preferir esquemas explícitos en producción: más rápido (evita un scan), más robusto ante datos malformados.

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

schema = StructType([
    StructField("producto", StringType(), nullable=True),
    StructField("cantidad", IntegerType(), nullable=False),
    StructField("precio", DoubleType(), nullable=True)
])
```

```
df = spark.read \  
    .schema(schema) \  
    .option("header", "true") \  
    .csv("ventas.csv")
```

`inferSchema=True` requiere un scan extra del archivo. En datasets grandes, siempre definir el esquema explícitamente (Apache Spark, 2024b)

03

Consultas y Análisis de Datos

SQL nativo, DataFrame DSL, joins y funciones avanzadas

API SQL: vistas temporales

Dos alcances de visibilidad

```
# Vista temporal local (solo en esta SparkSession)
df.createOrReplaceTempView("ventas")

resultado = spark.sql("""
    SELECT producto,
           SUM(cantidad * precio) AS total_revenue
    FROM   ventas
    WHERE  precio > 50
    GROUP  BY producto
    ORDER  BY total_revenue DESC
""")
resultado.show()
```

```
# Vista temporal global (compartida entre sesiones)
df.createGlobalTempView("ventas_global")
spark.sql("SELECT * FROM global_temp.ventas_global").show()
```

(Apache Spark, 2024a)

API DataFrame DSL

Encadenamiento de transformaciones

```
from pyspark.sql import functions as F

resultado = df \
    .filter(F.col("precio") > 50) \
    .withColumn("revenue", F.col("cantidad") * F.col("precio")) \
    .groupBy("producto") \
    .agg(
        F.sum("revenue").alias("total_revenue"),
        F.count("*").alias("num_ventas"),
        F.avg("precio").alias("precio_promedio")
    ) \
    .orderBy(F.desc("total_revenue")) \
    .limit(10)
```

SQL y el DSL producen el **mismo plan lógico** y el **mismo rendimiento** — Catalyst los optimiza igual. Elegir según legibilidad del equipo (Damji et al., 2020)

Joins en Spark SQL

Tipos y estrategias de ejecución

```
# Join con DSL
ventas.join(productos, on="producto_id", how="inner")
ventas.join(productos, on="producto_id", how="left")
ventas.join(clientes_vip, on="cliente_id", how="left_semi")    # EXISTS
ventas.join(clientes_vip, on="cliente_id", how="left_anti")    # NOT EXISTS

# Broadcast join – fuerza la estrategia (tabla pequeña en memoria)
from pyspark.sql.functions import broadcast
ventas.join(broadcast(dim_pais), on="pais_id")
```

Fuentes de Datos

Lectura y escritura unificada

```
# Lectura genérica
df = spark.read.format("parquet").load("s3://bucket/datos/")
df = spark.read.format("json").option("multiLine", "true").load("logs/")
df = spark.read.format("csv").option("header", "true").load("ventas.csv")
```

```
# JDBC – bases de datos relacionales
df_jdbc = spark.read.format("jdbc") \
    .option("url",      "jdbc:postgresql://host:5432/db") \
    .option("dbtable",  "public.ventas") \
    .option("user",     "user") \
    .option("password", "pass") \
    .load()

# Escritura – modos: overwrite, append, ignore, errorIfExists
df.write.format("parquet") \
    .partitionBy("año", "mes") \
    .mode("overwrite") \
    .save("output/ventas_parquet/")
```

(Apache Spark, 2024f, 2024g)

04 Ejemplos de Análisis con Spark SQL

UDFs, análisis exploratorio y optimización de consultas

UDFs — User-Defined Functions

Extender Spark SQL con lógica personalizada

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Definir la función Python
def clasificar_precio(precio):
    if precio is None: return "sin_dato"
    if precio < 50:     return "economico"
    if precio < 200:    return "medio"
    return "premium"
```

```
# Registrar como UDF
clasificar_udf = udf(clasificar_precio, StringType())

# Uso en DataFrame DSL
df.withColumn("segmento", clasificar_udf(F.col("precio")))

# Registrar para usar en spark.sql()
spark.udf.register("clasificar_precio", clasificar_precio, StringType())
spark.sql("SELECT producto, clasificar_precio(precio) FROM ventas")
```

Las UDFs en Python rompen la optimización de Catalyst y requieren serialización Python ↔ JVM. Preferir siempre **funciones built-in** de `pyspark.sql.functions` (Apache Spark, 2024h)

Análisis Exploratorio con Spark SQL

Comandos esenciales para entender un dataset

```
# Resumen estadístico
df.describe("precio", "cantidad").show()
df.summary("count", "mean", "stddev", "min", "25%", "75%", "max").show()

# Detectar nulos
from pyspark.sql import functions as F
df.select([F.count(F.when(F.col(c).isNull(), c)).alias(c)
           for c in df.columns]).show()
```

```
# Distribución de frecuencias
df.groupBy("categoria") \
  .count() \
  .withColumn("pct", F.round(F.col("count") / df.count() * 100, 2)) \
  .orderBy(F.desc("count")) \
  .show()

# Correlaciones numéricas
print(df.stat.corr("precio", "cantidad"))
df.stat.crosstab("categoria", "segmento").show()
```

(Damji et al., 2020, caps. 4–5)

05

Casos de Estudio

Cómo Airbnb, Uber y Netflix usan Spark SQL a escala

Airbnb

Pipeline de datos con Spark SQL y Parquet

El problema

- Miles de millones de eventos diarios de búsqueda, reserva y valoración
- Necesidad de analítica ad-hoc por equipos de producto y ciencia de datos

La solución

- Pipeline ETL con Spark SQL sobre datos en **Parquet particionado** por fecha
- **Apache Superset** como capa de visualización sobre Spark SQL
- Tablas Hive como metastore compartido

Resultados clave

- Tiempo de query reducido de horas a minutos
- Democratización del acceso a datos sin depender de ingeniería
- Reutilización de transformaciones como vistas SQL versionadas

Lección aprendida

Parquet columnar + predicado pushdown → **lectura selectiva de columnas y particiones** (Lu et al., 2018)

Uber

Hive sobre Spark SQL para métricas operativas

Contexto

- 500+ millones de viajes procesados diariamente
- Equipos de Data Science, Operaciones y Finanzas
- Necesidad de latencia baja en consultas OLAP

Arquitectura

- **Hadoop HDFS** como almacenamiento primario
- **Apache Hive** como metastore (esquemas centralizados)
- **Spark SQL** como motor de ejecución sobre Hive

Semantic Layer

- Métricas de negocio (viajes, ingresos, tiempos) definidas como vistas SQL
- Consistencia entre equipos: misma definición de "viaje completado"
- Versionado de métricas con Git

Lección aprendida

El **semantic layer** sobre Spark SQL elimina la fragmentación de métricas entre equipos (Mostak, 2026)

Netflix

Spark SQL + Apache Iceberg para datos a escala

Contexto

- 300+ millones de suscriptores generando telemetría de visualización
- Pipelines de ML para personalización de recomendaciones
- Datos históricos de años en producción

Herramientas

- **Apache Iceberg** como formato de tabla transaccional sobre S3
- **Spark SQL** para queries analíticas sobre Iceberg
- Time-travel queries para comparar estados históricos

Capacidades de Iceberg con Spark SQL

```
-- Consulta a versión histórica (time travel)
SELECT *
FROM netflix.viewing_history
TIMESTAMP AS OF '2025-12-01';

-- Compactación de small files
CALL netflix.system.rewrite_data_files(
  table => 'viewing_history'
);
```

Lección aprendida

Iceberg + Spark SQL resuelve el problema de **small files** y permite **schema evolution** sin downtime (Monte Carlo Data, 2024)

¡Gracias!

REFERENCIAS

Referencias

Referencias (1/2)

Artículos y libros

Apache Spark. (2024a). *Spark SQL, DataFrames and Datasets Guide*.

<https://spark.apache.org/docs/latest/sql-programming-guide.html>

Apache Spark. (2024b). *Data Types — Spark SQL Reference*.

<https://spark.apache.org/docs/latest/sql-ref-datatypes.html>

Apache Spark. (2024c). *Encoder — Spark 4.1.1 JavaDoc*.

<https://spark.apache.org/docs/latest/api/java/org/apache/spark/sql/Encoder.html>

Apache Spark. (2024d). *Window Functions — Spark SQL Reference*.

<https://spark.apache.org/docs/latest/sql-ref-syntax-qry-select-window.html>

Apache Spark. (2024e). *SQL Reference — Spark 4.1.1.*

<https://spark.apache.org/docs/latest/sql-ref.html>

Apache Spark. (2024f). *Generic Load/Save Functions.*

<https://spark.apache.org/docs/latest/sql-data-sources-load-save-functions.html>

Apache Spark. (2024g). *JDBC To Other Databases.* <https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html>

Apache Spark. (2024h). *Scalar User Defined Functions (UDFs).*

<https://spark.apache.org/docs/latest/sql-ref-functions-udf-scalar.html>

Apache Spark. (2024i). *Performance Tuning — Spark SQL.*

<https://spark.apache.org/docs/latest/sql-performance-tuning.html>

Referencias (2/2)

Artículos académicos y recursos

Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A., & Zaharia, M. (2015). Spark SQL: Relational data processing in Spark. *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 1383–1394. <https://doi.org/10.1145/2723372.2742797>

Damji, J. S., Wenig, B., Das, T., & Lee, D. (2020). *Learning Spark: Lightning-fast data analytics* (2nd ed.). O'Reilly Media. <https://pages.databricks.com/rs/094-YMS-629/images/LearningSpark2.0.pdf>

Databricks. (2015a). *Deep dive into Spark SQL's Catalyst Optimizer*.

<https://www.databricks.com/blog/2015/04/13/deep-dive-into-spark-sqls-catalyst-optimizer.html>

Databricks. (2016). *A tale of three Apache Spark APIs: RDDs, DataFrames, and Datasets*.

<https://www.databricks.com/blog/2016/07/14/a-tale-of-three-apache-spark-apis-rdds-dataframes-and-datasets.html>

Databricks. (2024). *Tutorial: Load and transform data using Apache Spark DataFrames*.

<https://docs.databricks.com/aws/en/getting-started/dataframes>

Engle, C., Lupper, A., Xin, R., Zaharia, M., Franklin, M. J., Shenker, S., & Stoica, I. (2013).

Shark: SQL and rich analytics at scale. *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, 13–24. <https://doi.org/10.1145/2463676.2465288>

Laskowski, J. (s.f.). *The internals of Spark SQL*. <https://books.japila.pl/spark-sql-internals/>

Lu, J., & Tang, L. (2018). *Building a data product based on Apache Spark at Airbnb*.

Databricks. <https://www.slideshare.net/databricks/building-data-product-based-on-apache-spark-at-airbnb-with-jingwei-lu-and-liyin-tang>

Monte Carlo Data. (2024). *Data engineering architecture: How they handle 500B events daily*.

<https://www.montecarlodata.com/blog-data-engineering-architecture/>

Mostak, T. (2026). *Secrets of the semantic layer in Big Tech*. Towards AI.

<https://pub.towardsai.net/secrets-of-the-semantic-layer-in-big-tech-how-uber-netflix-and-airbnb-manage-metrics-1b9f7680ac25>

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65. <https://doi.org/10.1145/2934664>